

1) What is Spring and its features?

Spring is a Java framework used to build enterprise-level applications easily and efficiently.

Main features:

1. **IoC (Inversion of Control)**: Manages object creation and dependency injection.
2. **AOP (Aspect-Oriented Programming)**: Helps with cross-cutting concerns like logging or security.
3. **Spring MVC**: Builds web applications using the Model-View-Controller pattern.
4. **Transaction Management**: Handles database transactions smoothly.
5. **Spring Boot**: Simplifies setup and configuration for faster development.
6. **Security**: Provides built-in authentication and authorization.

2) Which are the Spring framework modules

1. **Core Module** – Provides the core features like IoC and Dependency Injection.
2. **Bean Module** – Manages the creation and configuration of beans.
3. **Context Module** – Provides a way to access objects in a framework-style.
4. **Expression Language (SpEL)** – Used to query and manipulate objects at runtime.
5. **JDBC Module** – Simplifies database connectivity and error handling.
6. **ORM Module** – Integrates with Hibernate, JPA, etc.
7. **OXM Module** – Supports Object-to-XML mapping.
8. **JMS Module** – For sending and receiving messages via Java Messaging Service.
9. **Transaction Module** – Manages transactions in a consistent way.
10. **Web Module** – Supports basic web application features.
11. **Web-Servlet (Spring MVC)** – For web applications using the MVC pattern.

3) Explain the Core Container (Application context) module

It is the **main part of the Spring framework** that manages the **beans (objects)** and their **dependencies**.

Key points:

- Provides **IoC (Inversion of Control)** and **Dependency Injection**.
- The **ApplicationContext** interface is the central part — it loads bean definitions and wires them together.
- It also manages the **lifecycle** of beans.

In short: It's the **heart of Spring**, responsible for creating, configuring, and managing all beans in the application.

4) What is BeanFactory?

BeanFactory is the **basic container** in Spring that **manages beans (objects)**.

It is responsible for **creating, configuring**, and **providing** beans when needed.

Key points:

- It uses **lazy loading** (bean created only when requested).
- It supports **Dependency Injection**.
- It is the **parent interface** of `ApplicationContext`.

👉 **In short:** **BeanFactory** is the **core container** in Spring that manages and provides beans on demand.

5) What is XMLBeanFactory?

XMLBeanFactory is a **class in Spring** that reads bean definitions from an **XML configuration file** and creates beans based on it.

Key points:

- It's an **implementation of BeanFactory**.
- It **loads beans lazily** (only when requested).
- Used mainly in **older versions** of Spring.
- Now **deprecated** — replaced by **ApplicationContext** (like `ClassPathXmlApplicationContext`).

In short: XMLBeanFactory loads bean definitions from an XML file and manages them — but it's outdated now.

6) What is ApplicationContext in Spring, and how is it different from BeanFactory?

ApplicationContext is an advanced container in Spring that manages the lifecycle and dependencies of beans.

Difference:

- It extends BeanFactory with extra features.
- Supports eager loading, event handling, internationalization (i18n), and annotation-based configuration.

In short: ApplicationContext is a more powerful and commonly used version of BeanFactory.

7) What is Bean?

A bean in Spring is simply a Java object that is managed by the Spring container.

Key points:

- Beans are created, configured, and managed by Spring.
- They are defined in XML, Java configuration, or using annotations.
- The container controls their lifecycle and dependencies.

In short: A bean is an object created and managed by Spring, not by you.

8) What is Inversion of Control (IoC)?

It means **the control of object creation and management is given to the Spring container**, not done manually by the programmer.

Simple example:

Instead of you creating an object yourself, Spring creates it and gives it to you when needed.

In short: Spring controls how objects are created, connected, and managed — not you.

9) What is Inversion of Control (IoC)? What is Loose Coupling?

IoC: It means giving the control of object creation and dependency management to the Spring container instead of handling it manually.

Loose Coupling: It means classes depend on interfaces, not concrete implementations — making code easier to modify and test.

In short: IoC achieves loose coupling by letting Spring create and manage objects for you.

10) How to achieve IoC? What is the difference between Principle & Design Pattern?

How to achieve IoC:

IoC is achieved through Dependency Injection (DI) — Spring injects required dependencies automatically.

Principle vs Design Pattern:

- IoC is a principle (concept).
- DI is a design pattern used to implement that principle.

11) What is Dependency Injection (DI)? How to implement it?

Dependency Injection:

It means injecting dependent objects (dependencies) into a class from outside instead of creating them inside the class.

Implementation:

- Using XML configuration
- Using Annotations (@Autowired, @Component)
- Using Java-based configuration (@Configuration, @Bean)
-

12) What are the must-to-know things before learning DI in Spring?

You should understand:

- What beans are.

- How Spring container (ApplicationContext) works.
- How IoC manages object creation and wiring.
- Basic XML or annotation-based configuration.

13) How to implement Dependency Injection in code using Spring?

You can implement DI in three ways:

1. Constructor Injection – Dependencies are passed through the constructor.
2. Setter Injection – Dependencies are passed via setter methods.
3. Field Injection – Dependencies are injected directly using @Autowired.

14) What are the 4 ways of implementing DI in Spring?

1. Constructor Injection
2. Setter Injection
3. Field Injection
4. Interface Injection (*less commonly used in Spring, but conceptually valid*)

15) When to use which type of Dependency Injection?

- Constructor Injection: When dependency is mandatory or immutable.
- Setter Injection: When dependency is optional or can change later.
- Field Injection: Quick and simple, but not recommended for large or testable code.

16) Explain the AOP module?

The AOP module in Spring allows you to separate cross-cutting concerns — like logging, security, or transaction management — from your main business logic.

Key points:

- Helps add common behavior to multiple methods without repeating code.

- Uses aspects, advice, pointcuts, and join points to define where and when extra logic runs.
- Makes code cleaner, modular, and easier to maintain.

In short: AOP lets you apply common functionalities (like logging or security) across your app without changing the core code.

17) Difference between application context and bean factory?

1. **BeanFactory** is the basic container in Spring. It only provides the fundamental feature of creating and managing beans. Beans are created **lazily**, meaning the object is created *only when it is requested*. It uses less memory and is lightweight, but it does not support advanced features by default. It is mainly used in small or simple applications where you just need basic dependency injection.
2. **ApplicationContext** is an advanced container and a superset of BeanFactory. It creates all singleton beans **eagerly** at startup by default. It supports many enterprise-level features like event handling, internationalization (i18n), annotation support, AOP integration, and automatic BeanPostProcessor detection. It is commonly used in real-world Spring applications and enterprise projects.

18) what is ResponseEntity<>?

ResponseEntity is used in Spring to send a full HTTP response from a controller method. It allows you to send not just data, but also control the **status code** (like 200, 201, 404, 500) and **headers**.

Instead of returning only an object, ResponseEntity makes your API responses more meaningful and professional. For example, you can return success, failure, or not-found responses clearly.

Interview line:

ResponseEntity is used to control response data, status, and headers in one object.

19) Role of Component and @Component annotation in Spring

The @Component annotation is used to mark a Java class as a Spring-managed component. It indicates that the class is eligible for auto-detection and registration as a bean when component scanning is enabled. This means that Spring will automatically create an instance of this class and manage its lifecycle within the IoC container. In short, @Component tells Spring, "this class should be automatically detected and managed."

20) What are Bean and @Bean annotations? Difference between Bean and Component

A **Bean** in Spring is any object that is managed by the Spring IoC container.

The @Bean annotation is used inside configuration classes to define and register a bean manually. It is a method-level annotation that tells Spring to call that method and use its return value as a bean.

The difference between @Bean and @Component is that @Bean is used for explicit bean definitions, often for third-party or external classes where we cannot add annotations, while @Component is used for classes we create ourselves and can annotate directly for automatic detection.

21) Life cycle of a Bean in Spring or How Spring performs Dependency Injection internally

"Spring Bean lifecycle is the process from bean creation to destruction."

"Spring uses IoC container to create objects, find their dependencies, and inject them using constructor, setter, or field injection."

Steps:

1. Spring loads bean definition.
2. Bean object is created.
3. Dependencies are injected.
4. @PostConstruct / afterPropertiesSet() / init-method are called.
5. Bean is ready for use.
6. On container shutdown, @PreDestroy / destroy() / destroy-method are called.

One-Line Version (Very Short)

"Spring creates the bean, injects dependencies, runs initialization methods, makes it ready, and calls destroy methods when the container stops."

22) Externalizing Configuration in Spring Boot Interview Answer (Simple)

"Spring Boot lets us move configuration outside the code using properties or YAML files so we can change values without recompiling."

Method

Example

application.properties

server.port=8081

application.yml

server:\n port: 8081

OS Environment Variables

DB_PASSWORD

Command-line Args

--server.port=9090

Profiles

application-dev.properties

23) Most Common Spring Boot Actuator Endpoints.

Endpoint	Purpose
/actuator/health	App health status
/actuator/info	Application info
/actuator/metrics	JVM & app metrics
/actuator/beans	List all beans
/actuator/env	Environment properties
/actuator/mappings	All URLs
/actuator/loggers	Logging config
/actuator/shutdown	Stop application
/actuator/threaddump	Thread state
/actuator/heapdump	Memory dump

24) Difference between @Autowired and @Inject

@Autowired	@Inject
Spring Framework specific annotation	Java standard annotation (JSR-330)
Provided by Spring	Provided by javax.inject
Supports required = false	Does not support optional injection
Supports @Qualifier	Uses @Named instead of @Qualifier
Can inject by type or by name	Mainly injects by type
Better integrated with Spring features	Framework-independent (not Spring specific)

@Autowired is Spring-specific and more flexible, while @Inject is a Java standard annotation used for dependency injection.

25) How @Transactional works internally (Propagation + Isolation)

@Transactional uses Spring AOP and proxies to start, manage, and commit/rollback transactions, while propagation controls transaction joining and isolation controls data visibility.

26) What is AppConfig, @Configuration, and @ComponentScan

AppConfig is a common name used for a class that holds Spring configuration.

The @Configuration annotation marks a class as a configuration class that defines one or more beans. It tells Spring that the class contains methods or annotations defining bean definitions.

The @ComponentScan annotation tells Spring which packages to scan for classes annotated with @Component, @Service, @Repository, or @Controller. This enables automatic detection and registration of beans without explicitly defining them in configuration files.

27) What is the Spring IoC container? What is Annotation ConfigApplicationContext

The Spring IoC (Inversion of Control) container is responsible for creating, managing, and injecting dependencies between beans. It reads bean definitions, instantiates beans, wires them together, and manages their complete lifecycle.

Annotation ConfigApplicationContext is a type of Spring container that uses Java-based configuration. It reads configuration classes annotated with @Configuration and registers the beans defined within them. It is a

modern replacement for XML-based configuration and is commonly used in annotation-driven Spring applications.

28) Role of @Autowired annotation

The @Autowired annotation is used for automatic dependency injection. It allows Spring to automatically resolve and inject collaborating beans into a class. By default, it works by type, meaning Spring looks for a matching bean in the container that fits the required type and injects it automatically.

@Autowired can be applied to constructors, fields, or setter methods. If multiple beans of the same type exist, the developer can use @Qualifier to specify which bean should be injected.

29) What is the scope of a bean? What is the default scope?

Scope defines **how many instances** of a bean Spring creates and **how long** those instances live. The **default scope** in Spring is **Singleton**, meaning only **one shared instance** is created per Spring container.

30) What are the different types of Spring bean scopes? What is the @Scope annotation?

Common Spring scopes:

- **Singleton** – One instance per container (default).
- **Prototype** – New instance every time it is requested.
- **Request** – One instance per HTTP request (web apps).
- **Session** – One instance per HTTP session (web apps).

- **Application** – One instance per ServletContext.

@Scope is an annotation used to **specify which scope** a bean should follow.

31) What is Singleton Scope?

In **Singleton scope**, Spring creates **only one instance** of the bean. All requests for that bean return the **same object**, making it memory-efficient and suitable for stateless components like services.

32) What are circular dependencies? How does Spring handle them?

A **circular dependency** happens when Bean A depends on Bean B, and Bean B also depends on Bean A. Spring handles this in two ways:

- For **singleton beans**, it can resolve them using **early references**.
 - For **constructor injection**, circular dependencies usually fail, and Spring throws an error.
-

33) What is Spring Boot? How does it differ from the traditional Spring Framework?

Spring Boot is a framework built on top of Spring that **simplifies application development**.

It removes the need for complex XML configuration and manual setup.

Key Differences from Traditional Spring:

- Provides **auto-configuration**, reducing manual bean setup.
- Comes with **embedded servers** (Tomcat, Jetty).

- Uses **starter dependencies** for faster setup.
 - Offers **opinionated defaults** to follow standard Spring patterns.
 - Enables faster development suitable for microservices.
-

34) How does Spring Boot provide auto-configuration or How does Spring Boot auto-configuration determine which beans to create?

Spring Boot scans the classpath and automatically configures beans based on:

- Available dependencies
- Set properties in application.yml/properties
- Default Spring configurations

It uses the @EnableAutoConfiguration mechanism internally to **create and wire beans automatically**, saving developer effort.

35) What is the role of @SpringBootApplication annotation?

@SpringBootApplication is a **combination of three annotations**:

- @Configuration (marks config class)
- @EnableAutoConfiguration (enables auto setup)
- @ComponentScan (scans for components)

It acts as the **main entry point** for any Spring Boot application.

36) What is the project structure of a Spring Boot MVC application?

A typical structure includes:

- **Controller layer:** Handles incoming requests

- **Service layer:** Business logic
- **Repository layer:** Database access
- **Model/Entity:** Data representation
- **Resources:** Templates, static files, application properties
- **Main Class:** Bootstraps the application

This layered structure supports clear separation of concerns.

37) What is Spring MVC? Explain its architecture.

Spring MVC is a framework for building web applications following the **Model-View-Controller** pattern.

Architecture Flow:

1. Request comes to **DispatcherServlet**
2. It forwards to **Controller**
3. Controller processes and returns a model
4. DispatcherServlet sends the model to a **ViewResolver**
5. View (HTML/JSP/JSON) is returned to the user

It provides clean separation between UI, logic, and data.

38) What is the role of DispatcherServlet or Front Controller in Spring MVC?

DispatcherServlet is the **front controller** of Spring MVC. Its responsibilities:

- Receive all incoming requests
- Route requests to the appropriate controller

- Manage view resolution
- Integrate with components like handler mappings, interceptors, etc.

It is the **central coordinator** for the entire MVC flow.

39) Difference between @Controller and @RestController

@Controller

- Used for **web pages** (returns views like JSP/HTML).
- Requires @ResponseBody to return data.

@RestController

- Used for **REST APIs**.
- Automatically returns JSON/XML without needing @ResponseBody.

So, @RestController = @Controller + @ResponseBody.

40) Difference between @RequestParam and @PathVariable

@RequestParam

- Extracts values from **query parameters**
- Example: /user?id=10

@PathVariable

- Extracts values from the **URL path**
- Example: /user/10

Use @RequestParam for optional filters; @PathVariable for identifying resources.

41) Difference between @Component, @Service, @Repository, @Controller

Answer (short and perfect for interview):

All four are specializations of @Component, meaning they are Spring-managed beans.

The differences are based on **layer and purpose**:

@Component is a general-purpose annotation used to mark any Java class as a Spring-managed object (bean). If you don't know what to choose among other annotations, you can use @Component. It means "this is a normal Spring class" with no special role.

@Service is used for **business logic** classes. It tells Spring that this class contains the main logic of the application, like calculations, validations, or processing data. It works like @Component internally, but gives meaning that the class is part of the service layer.

@Repository is used for **database-related classes** (DAO layer). It handles operations like saving, fetching, updating, and deleting data from DB. It also helps Spring handle database errors properly by converting them into Spring exceptions.

@Controller is used for **handling web requests**. It receives browser or API requests and returns a web page or response. It connects the frontend with backend logic. For REST APIs, we usually use @RestController instead.

In short:
All are beans, but each annotation represents a specific application layer.

42) What is HandlerMapping in Spring MVC?

Answer:

HandlerMapping determines **which controller method should handle an incoming HTTP request.**

When a request arrives, DispatcherServlet asks HandlerMapping: "Which controller should process this URL?"

It maps:

- URL → Controller
- HTTP Method → Handler method

Example: /users/10 → UserController.getUser()

43) What is ViewResolver in Spring MVC?

Answer:

ViewResolver decides **which view (HTML, JSP, Thymeleaf, etc.) should be returned** after the controller processes a request.

It takes:

- View name (like "home")
and returns:
 - Actual view file (/WEB-INF/views/home.jsp, or home.html, etc.)

In short:

ViewResolver maps a view name to the actual UI page.

44) What is @InitBinder in Spring MVC?

Answer:

@InitBinder is used inside a controller to **customize data binding** for request parameters.

Use cases:

- Convert String → Date
- Format numbers
- Apply validation rules

It helps control how form data is converted to Java objects.

45) What is BeanPostProcessor?

Answer:

BeanPostProcessor allows you to run logic **before and after a bean is initialized**.

It has two key methods:

- postProcessBeforeInitialization()
- postProcessAfterInitialization()

Spring uses it internally for:

- @Autowired processing
- @PostConstruct
- AOP proxies

In short: It lets Spring modify or wrap beans after they are created but before they are used.

46) What is BeanFactoryPostProcessor?

Answer:

BeanFactoryPostProcessor allows you to modify **bean definitions** before the beans are actually created.

It executes:

- After Spring reads bean configurations
- Before Spring creates the beans

Use cases:

- Modify property values
- Change bean metadata
- Externalize configurations

In

short:

It works on **bean definitions**, not on bean instances.

47) Difference between ApplicationContext and WebApplicationContext

Answer:

ApplicationContext

- General Spring container (used in standalone apps)
- Manages beans, DI, events, internationalization

WebApplicationContext

- Special version of ApplicationContext for web applications
- Associated with ServletContext
- Provides additional MVC components (controllers, view resolvers)

In short:

ApplicationContext is core; WebApplicationContext is web-specific and integrates with the web environment.

48) What is ObjectMapper?

ObjectMapper is a core class from **Jackson** library used to **convert between Java objects and JSON**.

It handles **serialization** (Java → JSON) and **deserialization** (JSON → Java).

Interview line:

“ObjectMapper is the main Jackson class used for converting Java objects to JSON and vice-versa.”

Key Methods of ObjectMapper (Short + Direct)

1. readValue()

- Converts **JSON → Java object**.
- Most commonly used.

Example Use:

```
objectMapper.readValue(jsonString, User.class)
```

Interview:

“readValue is used to deserialize JSON into Java POJOs.”

2. writeValue() / writeValueAsString()

- Converts **Java Object → JSON**.
- writeValueAsString() returns JSON as plain string.

Example:

```
objectMapper.writeValueAsString(user)
```

Interview:

“writeValueAsString converts a Java object into JSON string.”

3. readTree()

- Parses JSON into a **JsonNode tree model** (like a DOM for JSON).
- Best when structure is dynamic or unknown.
- You can traverse nodes using `.get()`, `.path()`, `.asText()` etc.

Example:

```
JsonNode node = objectMapper.readTree(jsonString);
```

Interview:

“readTree reads JSON into a JsonNode tree when you don’t want to bind it to a POJO.”

4. createObjectNode() & createArrayNode()

- Used to build JSON dynamically using `JsonNode`.

Interview:

“createObjectNode helps in programmatically building JSON objects.”

5. convertValue()

- Converts **one Java type to another Java type** using JSON under the hood.

Example:

```
UserDTO dto = objectMapper.convertValue(user, UserDTO.class);
```

Interview:

"convertValue transforms an object of one type to another without writing custom code."

6. treeToValue()

- Converts JsonNode → Java Object.

Interview:

"treeToValue maps a JsonNode back to a Java POJO."

7. valueToTree()

- Converts Java Object → JsonNode.

Interview:

"valueToTree converts a Java object into a JsonNode tree model."

8. enable() / disable()

- Used to configure features (e.g., ignore unknown fields, pretty print)

Common configs:

```
mapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);
```

Interview:

"enable/disable is used to customize serialization or deserialization behavior."

49) What is Pojo classes?

A **POJO (Plain Old Java Object)** is a simple Java class that **only contains fields, constructors, getters, setters, and basic methods**. It does **not** depend on any special framework, inheritance, or annotations. POJOs are used to represent **clean, lightweight data models** in applications—for example, entities, DTOs, and request/response objects.

50) What is a Logging Framework and @Slf4j annotation?

A **logging framework** (like Log4j2, SLF4J, Logback) helps applications record important runtime information such as errors, warnings, and debug messages. Logging helps with debugging, monitoring, and production issue analysis.

@Slf4j is a **Lombok annotation** that automatically generates an slf4j logger in your class, so you don't need to manually write private static final Logger log = LoggerFactory.getLogger(...). It makes logging cleaner and reduces boilerplate code.

51) What are Java Generics (Short Paragraph)?

Java Generics allow you to write **type-safe, reusable classes and methods** by using type parameters like <T>, <E>, or <K,V>. They ensure compile-time type checking and prevent ClassCastException by enforcing that collections and methods work with a specified data type, e.g., List<String>, Map<Integer, User>, or Optional<T>.

52) What is App Cache and how to make it using @PostConstruct?

App Cache means storing data in memory inside the application so it can be accessed quickly without repeatedly calling the database or

external APIs. You usually preload the data when the application starts. Using `@PostConstruct`, you can load this data into a cache (like a `Map` or `ConcurrentHashMap`) right after the Spring bean is initialized. This ensures the cache is ready before the application starts serving requests.

53) What is RedisTemplate?

`RedisTemplate` is a Spring Data Redis class used to interact with a Redis server for performing operations like saving, retrieving, deleting, and updating data. It abstracts low-level Redis commands and provides high-level methods to work with Redis data types such as Strings, Hashes, Lists, Sets, and Sorted Sets. It handles serialization/deserialization and connection management internally, so developers can store Java objects directly in Redis. Essentially, **`RedisTemplate` is the main tool in Spring for performing Redis caching and data operations in a type-safe and convenient way.**

54) @Lazy Annotation (Short Para)

`@Lazy` in Spring is used to **delay the creation of a bean** until it is actually needed. Normally Spring creates all beans at application startup (eager initialization), but when you mark a bean or injection point with `@Lazy`, Spring will create its instance only when it's first requested. This helps improve startup time, avoids unnecessary memory usage, and is useful for breaking **circular dependencies**. You can place `@Lazy` on a class, a specific dependency, or at configuration level to control when objects are initialized.